

1) 2017 Brand Revenue Leaderboard (Top 3)

Task: Return the **top 3 brands** by **net revenue** in calendar year **2017**.

Include: `brand_name` , `net_revenue` .

Rules:

- Revenue = `SUM(quantity * list_price * (1 - discount))`
- Only **Completed** orders (`order_status = 4`)
- Break ties fairly (e.g., include all tied for 3rd), and sort by revenue desc.

You'll need: multi-table JOINS (`order_items` → `orders` → `products` → `brands`), date filtering on `order_date` , computed expression with discount, `GROUP BY` + `HAVING` , and a top-N strategy without windows (e.g., `TOP WITH TIES` or `TOP 3` with careful tie handling).

2) Category-Diverse Customers (2017)

Task: List customers who purchased from **≥ 3 distinct categories** in **2017** (completed orders only).

Include: `customer_id` , customer full name, `distinct_category_count` .

Sort by `distinct_category_count desc`, then name.

You'll need: multi-join (`orders` → `order_items` → `products` → `categories`), `COUNT(DISTINCT ...)` , date filter on `order_date` , `GROUP BY` , and a `HAVING` threshold.

3) Late Shipments by Product (2016)

Task: Find products with **late shipments** in **2016** where `shipped_date > required_date` (completed orders only).

Return: `product_id`, `product_name`, `late_orders_count`, and `max_delay_days` (largest `DATEDIFF(day, required_date, shipped_date)` for that product).

Order by `late_orders_count desc`, then `max_delay_days desc`.

You'll need: JOINS (`orders` ↔ `order_items` ↔ `products`), date comparisons & `DATEDIFF`, filtered `WHERE`, `GROUP BY`, aggregate math, and ordering.

4) Manager Roll-ups (Team Size & Sales in 2017)

Task: For each **manager** (a staff member who has direct reports), show:

- `manager_id`, `manager_name`
- `direct_reports_count`
- `team_net_revenue_2017` = sum of **completed** orders in 2017 handled by the **manager's direct reports**, using `quantity * list_price * (1 - discount)`.

You'll need: **self-join** on `sales.staffs` via `manager_id`, JOIN to `orders` (by `staff_id`) and then `order_items`, date filtering, `GROUP BY`. Consider `active` flag if you want to limit to active staff.

5) Stock vs. Demand Gap (All Time)

Task: Identify any `product_id` where **total ordered quantity** (all time, across all stores) **exceeds total on-hand stock** recorded in `production.stocks` (sum across stores).

Return: `product_id`, `product_name`, `total_ordered_qty`, `total_stock_qty`,
`shortfall = total_ordered_qty - total_stock_qty` (only rows where `shortfall > 0`).

Sort by `shortfall desc`.

You'll need: aggregations from `order_items` , a separate aggregation from `production.stocks` , then **JOIN** or **subquery/CTE** to compare, plus `HAVING` on the computed shortfall.
